WORKSHOP PENGEMBANGAN WEBSITE LANJUT

LAPORAN MINGGU 6 / PERTEMUAN 1



Disusun Oleh:

E41263336

Ahmad Azizul Amin

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

2026

Acara 10

1.1 RegistrationForm.module.css

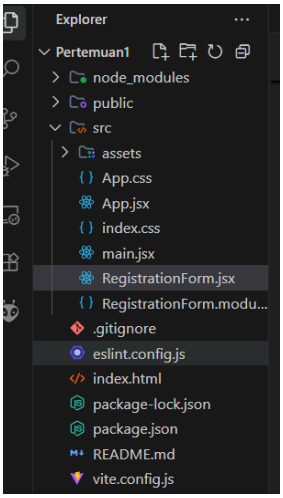
1. Membuat Project

```
PS F:\Teknik Informatika\Semester 1\Workshop Pengembangan Website Lanjut\Minggu_6\Pertemuan1> npm create vite@latest .

> npx
> create-vite .


◇  Select a framework:
│  React
│
◇  Select a variant:
│  JavaScript
│
◇  Which linter to use?
│  ESLint
```

Disini saya membuat project berbasis react menggunakan javascript dengan bantuan vite dan memasang Eslint untuk merapikan kodenya, di dalam project tersebut juga terdapat 2 file baru yaitu RegistrationFrom.module.css dan RegistrationForm.jsx. berikut hasilnya:



2. Membuat Container

```css
/* RegistrationForm.module.css */

* {
    color: black;
}

.container {
    max-width: 500px;
    margin: 2rem auto;
    padding: 2rem;
    background: white;
    border-radius: 12px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
}
```

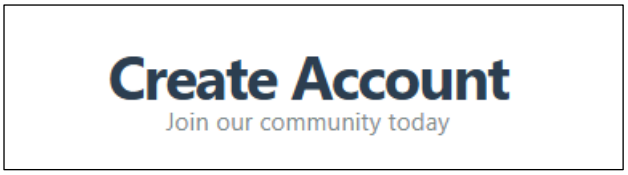
Kode diatas digunakan untuk membuat tampilan container agar di Tengah layar dengan background putih.

3. Membuat tampilan judul

```css
.title {
    text-align: center;
    color: #2c3e50;
    font-size: 2.5rem;
    margin-bottom: 0.5rem;
    font-weight: 700;
}

.subtitle {
    text-align: center;
    color: #7f8c8d;
    margin-bottom: 2rem;
    font-size: 1rem;
}
```

Kode ini untuk mengatur tampilan judul utama dan subjudulnya agar posisinya berada ditengah dengan warna yang berbeda. Hasilnya:



4. Membuat Styling Form

```css
.form {
    display: flex;
    flex-direction: column;
    gap: 1rem;
}

.formGroup {
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
}
```

Form disusun secara vertikal menggunakan display: flex dengan flex-direction: column dan diberi jarak antar elemen menggunakan gap.

```css
.label {
    font-size: 0.9rem;
    font-weight: 600;
    color: #2c3e50;
}
```

Label digunakan untuk memberikan informasi mengenai data yang perlu diisi pada setiap field, dengan menerapkan style seperti yang telah dijelaskan sebelumnya.

```css
.input {
    padding: 1rem;
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    font-size: 1rem;
    transition: all 0.3s ease;
    background: #fafafa;
}

.input:focus {
    outline: none;
    border-color: #3498db;
    background: white;
    box-shadow: 0 0 0 3px rgb(52, 152, 219, 0.1);
}

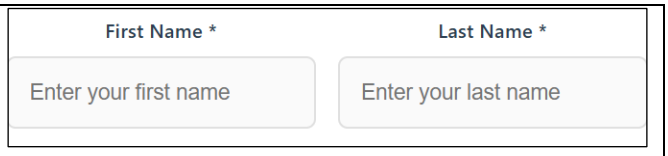
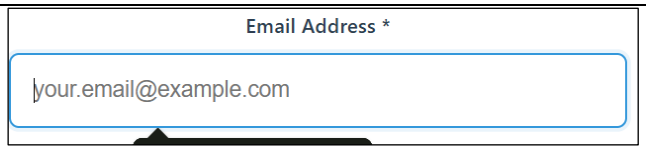
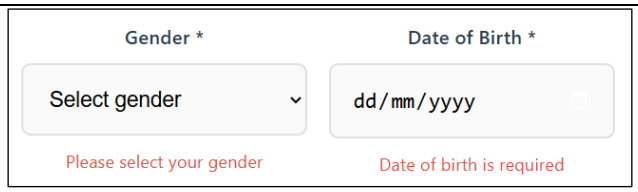
.input.eror {
    border-color: #e74c3c;
    background: #fdf2f2;
}
```

```css
.select {
    padding: 1rem;
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    font-size: 1rem;
    background: #fafafa;
    cursor: pointer;
}

.select.eror {
    border-color: #e74c3c;
    background: #fdf2f2;
}

.select:focus {
    outline: none;
    border-color: #3498db;
    background: white;
}
```

Field berupa input dan select dibuat dalam bentuk persegi panjang dengan padding yang cukup dan background berwarna #fafafa. Ketika field dipilih atau mendapatkan fokus, warna border akan berubah menjadi biru dan dilengkapi efek ring menggunakan box-shadow. Sementara itu, jika terdapat class error, border pada input akan berubah menjadi merah. Hasilnya:

State Normal	
State Focus	
State Error	

```css
.checkbox {
    width: 18px;
    height: 18px;
    cursor: pointer;
}
```

Pada checkbox, styling hanya diterapkan pada ukuran panjang dan lebarnya. Selain itu, ditambahkan cursor: pointer agar kursor berubah menjadi bentuk tangan saat diarahkan ke checkbox. Hasilnya:

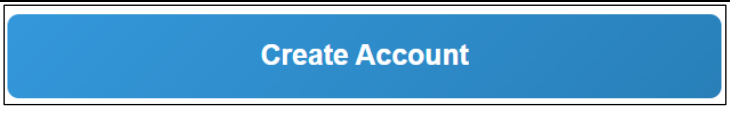
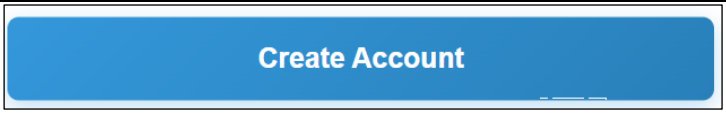
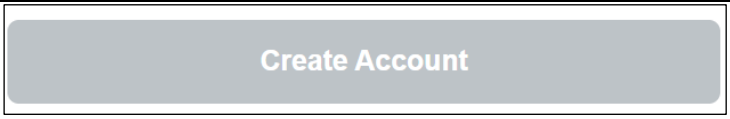



```css
.submitButton {
    padding: 1rem 2rem;
    background: linear-gradient(135deg, #3498db, #2980b9);
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 1.1rem;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    margin-top: 1rem;
}

.submitButton:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgb(52, 152, 219, 0.3);
}

.submitButton:disabled {
    background: #bdc3c7;
    cursor: not-allowed;
    transform: none;
    box-shadow: none;
}
```

Terakhir, formulir dilengkapi dengan styling pada tombol submit. Tombol ini menggunakan warna biru dengan teks berwarna putih. Ketika kursor diarahkan ke tombol, akan muncul efek box-shadow berwarna biru dan tombol sedikit terangkat untuk memberikan efek interaktif. Jika tombol dalam kondisi disabled, background akan berubah menjadi abu-abu dan tombol tidak dapat diklik. Hasil tampilannya dapat dilihat sebagai berikut:

State Normal	
State Hover	
State Disabled	

5. Membuat Styling success message

```css
.successMessage {
    text-align: center;
    color: #155724;
    font-size: 1rem;
    margin-top: 1rem;
    background: #d4edda;
    border: 1px solid #c3e6cb;
    border-radius: 8px;
    padding: 1rem;
}
```

Pesan berhasil akan ditampilkan dengan background berwarna hijau tua dan teks berwarna hijau agar terlihat jelas sebagai notifikasi keberhasilan.

6. Membuat responsiveness untuk device

```css
/* Responsive Design */
@media (max-width: 600px) {
    .container {
        margin: 1rem;
        padding: 1.5rem;
    }

    .title {
        font-size: 1.5rem;
    }
}
```

Kode di atas digunakan pada device dengan width di bawah atau sama dengan 600px yaitu mengubah style container dan title.

1.2 RegistrationForm.jsx

1. Import Package

```
src > RegistrationForm.jsx > RegistrationForm
1   import React, { useState } from "react";
2   import styles from "./RegistrationForm.module.css";
```

2. Inisialisasi state form

```jsx
const RegistrationForm = () => {
  const [formData, setFormData] = useState({
    firstName: "",
    lastName: "",
    email: "",
    password: "",
    confirmPassword: "",
    gender: "",
    dateOfBirth: "",
    phone: "",
    agreeToTerms: false,
  });

  const [errors, setErrors] = useState({});
  const [isSubmitted, setIsSubmitted] = useState(false);
```

3. Membuat fungsi handleInputChange

```
const handleInputChange = (e) => {
  const { name, value, type, checked } = e.target;
  setFormData((prev) => ({
    ...prev,
    [name]: type === "checkbox" ? checked : value,
  }));
  // Clear error when user starts typing
  if (errors[name]) {
    setErrors((prev) => ({
      ...prev,
      [name]: "",
    }));
  }
};
```

Pada kode tersebut, e.target digunakan untuk mengakses elemen yang sedang berinteraksi. Elemen input memiliki beberapa atribut seperti name, type, value, dan checked. Ketika nilai input berubah, state formData akan diperbarui berdasarkan atribut yang terdapat pada elemen tersebut.

4. Membuat fungsi handleSubmit

```
const handleSubmit = (e) => {
  e.preventDefault();

  const validationErrors = validateForm(formData);

  if (Object.keys(validationErrors).length === 0) {
    console.log("Form submitted:", formData);
    setIsSubmitted(true);
    // Reset form
    setFormData({
      firstName: "",
      lastName: "",
      email: "",
      password: "",
      confirmPassword: "",
      gender: "",
      dateOfBirth: "",
      phone: "",
      agreeToTerms: false,
    });
  } else {
    setErrors(validationErrors);
  }
};
```

Fungsi tersebut akan melakukan validasi menggunakan method validateForm. Jika tidak ditemukan error, state isSubmitted akan diubah menjadi true, kemudian formData akan dikembalikan ke kondisi awal. Namun, jika terdapat error, pesan error akan disimpan ke dalam state errors dan kemudian ditampilkan pada form.

5. Membuat method validationErrors

```
const validateForm = (data) => {

  if (!data.firstName.trim()) {
    errors.firstName = "First name is required";
  }

  if (!data.lastName.trim()) {
    errors.lastName = "Last name is required";
  }

  if (!data.email.trim()) {
    errors.email = "Email is required";
  }

  if (!data.password) {
    errors.password = "Password is required";
  }

  if (!data.confirmPassword) {
    errors.confirmPassword = "Please confirm your password";
  }

  if (data.password !== data.confirmPassword) {
    errors.confirmPassword = "Passwords do not match";
  }

  if (!data.gender) {
    errors.gender = "Please select your gender";
  }

  if (!data.dateOfBirth) {
    errors.dateOfBirth = "Date of birth is required";
  }

  if (!data.phone.trim()) {
    errors.phone = "Phone number is required";
```

Kode tersebut digunakan untuk memeriksa setiap field pada form untuk memastikan data yang dimasukkan sudah valid. Jika terdapat field yang tidak memenuhi ketentuan, maka akan ditampilkan pesan error sebagai informasi bagi pengguna.

6. Menampilkan view ketika state submitted

```
if (isSubmitted) {
  return (
    <div className={styles.container}>
      <div className={styles.successMessage}>
        <h3>🎉 Registration Successful!</h3>
        <p>
          Thank you for registering. We've sent a confirmation email to your
          inbox.
        </p>
      </div>
    </div>
  );
}
```

Pada tampilan, jika state submitted bernilai true, maka akan ditampilkan pesan keberhasilan.

7. Menampilkan view form

```
return (
  <div className={styles.container}>
    <h1 className={styles.title}>Create Account</h1>
    <p className={styles.subtitle}>Join our community today</p>

    <form className={styles.form} onSubmit={handleSubmit}>
      {/* Name Fields */}
      <div
        style={{
          display: "grid",
          gridTemplateColumns: "1fr 1fr",
          gap: "1rem",
        }}
      >
        <div className={styles.formGroup}>
          <label className={styles.label}>First Name *</label>
          <input
            type="text"
            name="firstName"
            value={formData.firstName}
            onChange={handleInputChange}
            className={`${styles.input} ${errors.firstName ? styles.error : ""
              }`}
            placeholder="Enter your first name"
          />
          {errors.firstName && (
            <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
              {errors.firstName}
            </span>
          )}
        </div>

        <div className={styles.formGroup}>
          <label className={styles.label}>Last Name *</label>
          <input
            type="text"
            name="lastName"
            value={formData.lastName}
            onChange={handleInputChange}
            className={`${styles.input} ${errors.lastName ? styles.error : ""
              }`}
            placeholder="Enter your last name"
          />
          {errors.lastName && (
            <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
              {errors.lastName}
            </span>
          )}
        </div>
      </div>

      {/* Email */}
      <div className={styles.formGroup}>
        <label className={styles.label}>Email Address *</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleInputChange}
          className={`${styles.input} ${errors.email ? styles.error : ""}`}
          placeholder="your.email@example.com"
        />
        {errors.email && (
          <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
            {errors.email}
          </span>
        )}
      </div>
```

```jsx
<div
  style={{
    display: "grid",
    gridTemplateColumns: "1fr 1fr",
    gap: "1rem",
  }}
>
  <div className={styles.formGroup}>
    <label className={styles.label}>Password *</label>
    <input
      type="password"
      name="password"
      value={formData.password}
      onChange={handleInputChange}
      className={`${styles.input} ${errors.password ? styles.error : ""
        }`}
      placeholder="Create a password"
    />
    {errors.password && (
      <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
        {errors.password}
      </span>
    )}
  </div>

  <div className={styles.formGroup}>
    <label className={styles.label}>Confirm Password *</label>
    <input
      type="password"
      name="confirmPassword"
      value={formData.confirmPassword}
      onChange={handleInputChange}
      className={`${styles.input} ${errors.confirmPassword ? styles.error : ""
        }`}
      placeholder="Confirm your password"
    />
    {errors.confirmPassword && (
      <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
        {errors.confirmPassword}
      </span>
    )}
  </div>
</div>

{/* Personal Information */}
<div
  style={{
    display: "grid",
    gridTemplateColumns: "1fr 1fr",
    gap: "1rem",
  }}
>
  <div className={styles.formGroup}>
    <label className={styles.label}>Gender *</label>
    <select
      name="gender"
      value={formData.gender}
      onChange={handleInputChange}
      className={styles.select}
    >
      <option value="">Select gender</option>
      <option value="male">Male</option>
      <option value="female">Female</option>
      <option value="other">Other</option>
    </select>
    {errors.gender && (
      <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
        {errors.gender}
      </span>
    )}
  </div>
```

```
    <div className={styles.formGroup}>
      <label className={styles.label}>Date of Birth *</label>
      <input
        type="date"
        name="dateOfBirth"
        value={formData.dateOfBirth}
        onChange={handleInputChange}
        className={`${styles.input} ${errors.dateOfBirth ? styles.error : ""
          }`}
      />
      {errors.dateOfBirth && (
        <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
          {errors.dateOfBirth}
        </span>
      )}
    </div>
  </div>

  {/* Phone */}
  <div className={styles.formGroup}>
    <label className={styles.label}>Phone Number *</label>
    <input
      type="tel"
      name="phone"
      value={formData.phone}
      onChange={handleInputChange}
      className={`${styles.input} ${errors.phone ? styles.error : ""}`}
      placeholder="+1 234 567 8900"
    />
    {errors.phone && (
      <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
        {errors.phone}
      </span>
    )}
  </div>
        {/* Terms and Conditions */}
        <div className={styles.formGroup}>
          <label className={styles.checkboxGroup}>
            <input
              type="checkbox"
              name="agreeToTerms"
              checked={formData.agreeToTerms}
              onChange={handleInputChange}
              className={styles.checkbox}
            />
            <span className={styles.checkboxLabel}>
              I agree to the Terms and Conditions and Privacy Policy
            </span>
          </label>
          {errors.agreeToTerms && (
            <span style={{ color: "#e74c3c", fontSize: "0.8rem" }}>
              {errors.agreeToTerms}
            </span>
          )}
        </div>

        {/* Submit Button */}
        <button
          type="submit"
          className={styles.submitButton}
          disabled={!formData.agreeToTerms}
        >
          Create Account
        </button>
      </form>
    </div>
  );
};

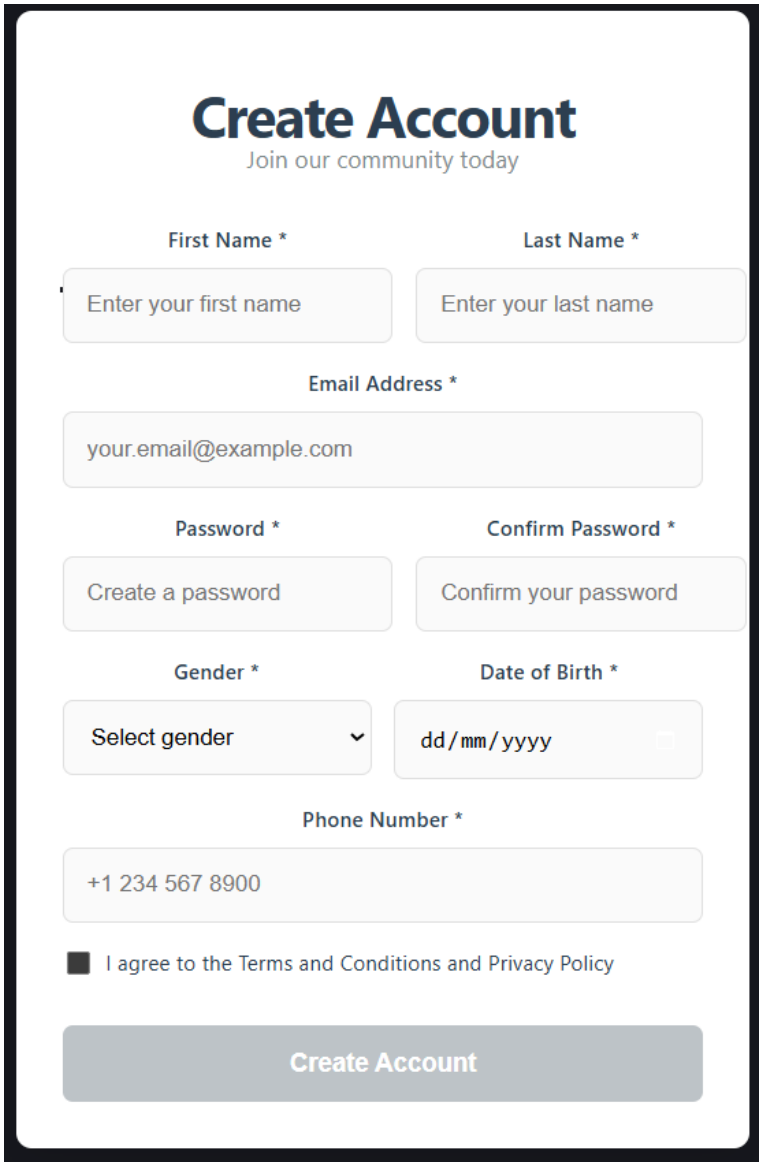
export default RegistrationForm;
```

Penjelasan:

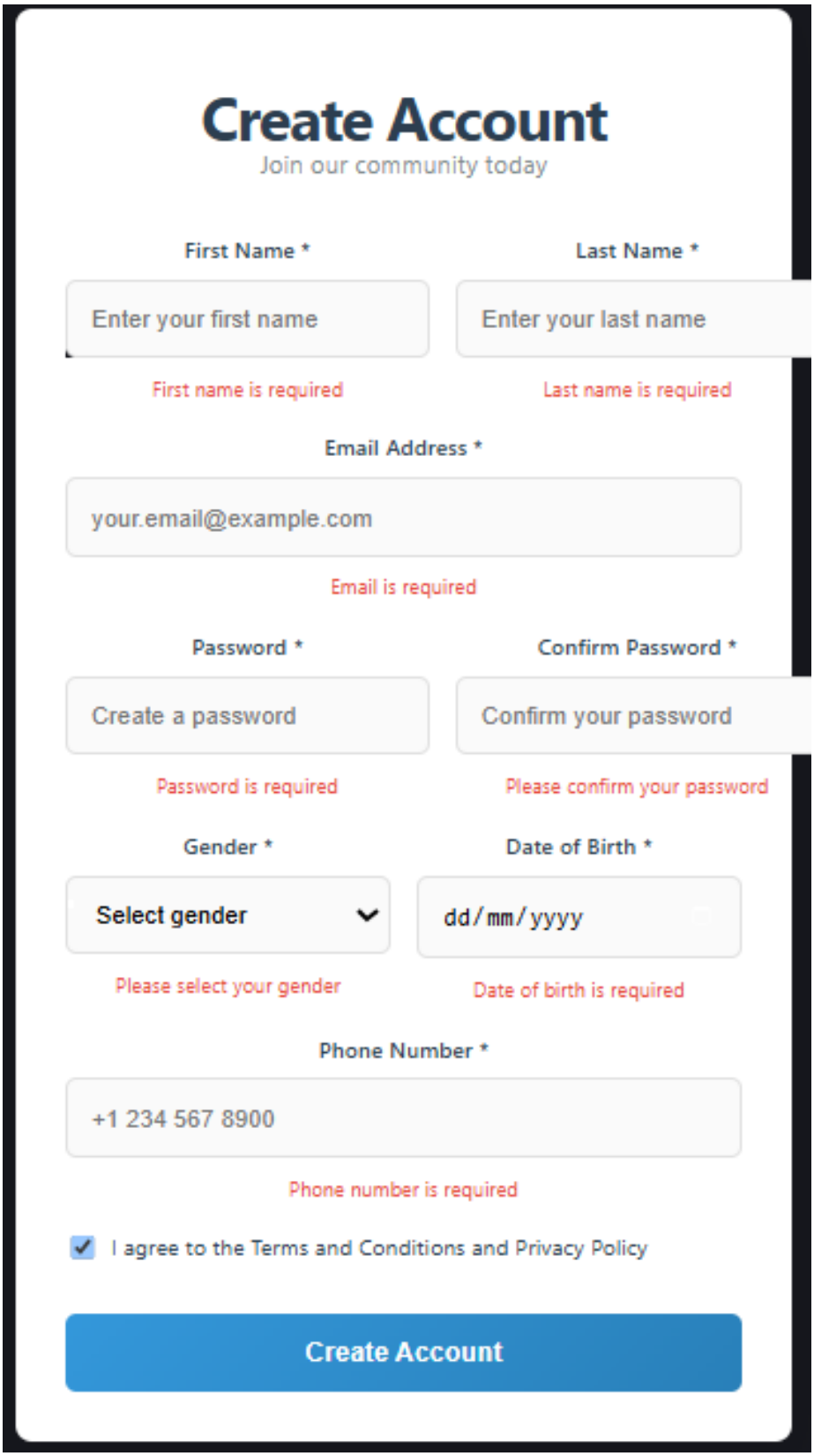
Kode tersebut mengimplementasikan beberapa field pada formulir, yaitu First Name, Last Name, Email, Password, Password Confirmation, Gender, Date of Birth, Phone, serta checkbox Agree to Terms. Pada field First Name, Last Name, Email, Password, Password Confirmation, Date of Birth, dan Phone digunakan komponen input dengan styling dari class input. Field Gender menggunakan komponen select dengan class select, sedangkan Agree to Terms menggunakan input type="checkbox" dengan class checkbox. Terakhir, tombol submit menggunakan komponen button dengan styling dari class submitButton.

1.3 Hasil Akhir

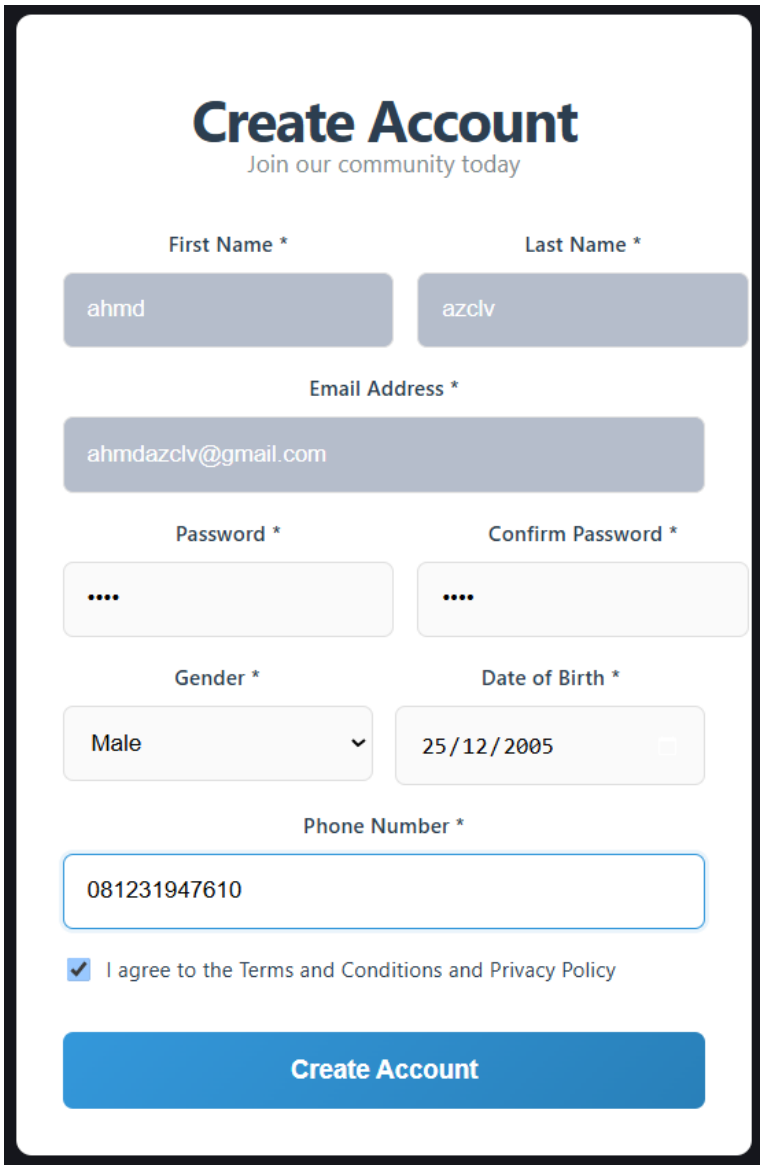
1. State Awal



2. State Eror


Create Account
Join our community today
First Name *
Last Name *
Enter your first name
Enter your last name
First name is required
Last name is required
Email Address *
your.email@example.com
Email is required
Password *
Confirm Password *
Create a password
Confirm your password
Password is required
Please confirm your password
Gender *
Date of Birth *
Select gender
dd/mm/yyyy
Please select your gender
Date of birth is required
Phone Number *
+1 234 567 8900
Phone number is required
I agree to the Terms and Conditions and Privacy Policy
Create Account

3. State Terisi



4. State Setelah Submit

